PROJECT REPORT

QUIZ MANAGEMENT SYSTEM

Project Code: CPP53

Enrollment Number: IU2541230517

A console-based C++ application demonstrating object-oriented programming, menu-driven question management, quiz scoring, and text-file persistence.

Prepared for academic project submission

1. Project Title

Quiz Management System

2. Student Details

Field	Details
Enrollment Number	IU2541230517
Project Code	CPP53
Implementation Language	C++ (C++11 or later)
Application Type	Console-based menu-driven application

3. Problem Statement

Quizzes are widely used to assess knowledge, but managing questions, answer options, student attempts, and scores manually can be repetitive and error-prone. A small quiz application can centralize these activities, provide a consistent question format, calculate scores automatically, and preserve records between runs. The Quiz Management System addresses this need

through a C++ console program that lets an administrator maintain a question bank and lets a student answer the available questions and view a score.

This implementation is an academic prototype. It uses text files for persistence and a fixed in-memory capacity, rather than a database or graphical user interface.

4. Objectives

- Develop a menu-driven C++ program for quiz question management.
- Support adding, displaying, searching, updating, and deleting questions.
- Allow a student to take a quiz and receive an automatically calculated score.
- Record student ID, score, and total question count for each attempt.
- Save and reload questions and attempt records using text files.
- Demonstrate encapsulation, constructors/destructors, inheritance, virtual functions, dynamic memory, file handling, and exception handling.
- Separate the implementation into header (.h) and source (.cpp) files.

5. Features

Menu operation	Description
1. Add Record (Question)	Collects an ID, question text, four options, and the correct option. Checks duplicate IDs, empty question text, and correct option range 1–4.
2. Display Records	Displays all stored questions and answer options.
3. Search Record	Searches for a question by ID.
4. Update Record	Updates the text of an existing question; options and correct answer remain unchanged.
5. Delete Record	Deletes a question by ID and shifts subsequent records.
6. Take Quiz	Collects student ID and name, displays each question, accepts answers, and calculates the score.
7. Report	Displays the question count, attempt count, and score records.
8. Save/Load Data	Saves current records to files and reloads them.
9. Exit	Saves data and ends the program.

At startup, main() constructs QuizSystem, calls loadData(), and then starts the menu. Question and attempt arrays each have a capacity of 100. Run the application from the project root so the relative paths Data/questions.txt and Data/attempts.txt resolve correctly.

6. Classes and Responsibilities

Class	Data members	Responsibilities and key functions
Person (abstract)	protected string name	Defines the common name field, constructors, virtual destructor, and pure virtual display() interface.
Student	private int studentId; inherited name	Represents a student. Provides constructors, getId(), getName(), and display().
Question	private id, text, options[4], correctOption	Stores one multiple-choice question. Provides constructors, getters, setText(), and display().
Quiz	private quizId, title	Stores the quiz ID and title; provides constructors, getters, and display().
Attempt	private studentId, score, total	Stores the result summary for one attempt; provides constructors, getters, and display().
FileManager	Static methods; no instance data	Saves and loads questions and attempts using text files.
QuizSystem	Question* questions; Attempt* attempts; counts; capacity; Quiz quiz	Coordinates the menu, CRUD operations, quiz process, report, dynamic arrays, and persistence calls.

The classes separate responsibilities: Question represents quiz content, Attempt represents a result, FileManager handles persistence, and QuizSystem coordinates the user workflow. Student publicly inherits Person.

7. OOP Concepts Used

7.1 Classes and Objects

Each class defines a type of entity. Question objects hold an ID, text, four options, and a correct option. In takeQuiz(), the program creates a Student object from the entered student ID and name. An Attempt object records the student's score and the number of questions.

7.2 Encapsulation

Data members are private where appropriate and accessed through public member functions. For example, Question keeps id, text, options, and correctOption private. Other parts of the program use getId(), getText(), getOption(), and getCorrectOption() instead of accessing these fields directly.

7.3 Constructors and Destructors

Classes provide default and parameterized constructors. Question() initializes a default object, while the parameterized Question constructor initializes it from supplied values. QuizSystem() allocates the question and attempt arrays. QuizSystem::~QuizSystem() releases both arrays with delete[].

7.4 Inheritance

Student publicly inherits Person. Person stores the shared name field and defines the display interface. Student adds studentId and overrides display(). This is an “is-a” relationship: a Student is a kind of Person.

7.5 Runtime Polymorphism

Person declares display() as a pure virtual function and has a virtual destructor. Student overrides display() using override. This supports runtime polymorphism through a Person pointer or reference. In the current workflow, Student is used directly, so virtual dispatch through a base-class pointer is supported by the design but not explicitly demonstrated.

7.6 Dynamic Memory Allocation

QuizSystem allocates arrays at runtime using questions = new Question[capacity] and attempts = new Attempt[capacity], with capacity set to 100. Its destructor calls delete[] for both arrays. The arrays have a fixed capacity and do not grow automatically.

7.7 Validation and Exception Handling

The menu loop uses try/catch to report exceptions, including file-save errors. addQuestion() checks duplicate IDs, empty question text, and correct-option values. The menu handles non-numeric menu input. Numeric input validation in some other operations remains basic.

8. File Handling

The program uses plain-text files in the Data directory. Each question is stored on one pipe-delimited line with seven fields: ID, question text, four options, and correct option number. Example format: 1|What is 2 + 2?|3|4|5|6|2. Since the vertical bar is the field separator, question text and options should not contain that character.

Each attempt is stored as three fields: student ID, score, and total questions. Student names are requested during a quiz but are not saved in the attempt file. FileManager uses ofstream to write data and ifstream/stringstream to read it.

Saving and Loading Process

- At startup, main() calls QuizSystem::loadData(), which loads Data/questions.txt and Data/attempts.txt.
- Menu option 8 calls saveData() and then loadData().
- Menu option 9 saves both files before exiting.
- Records are loaded up to the fixed capacity of 100; malformed question lines are skipped.

Persistence Testing

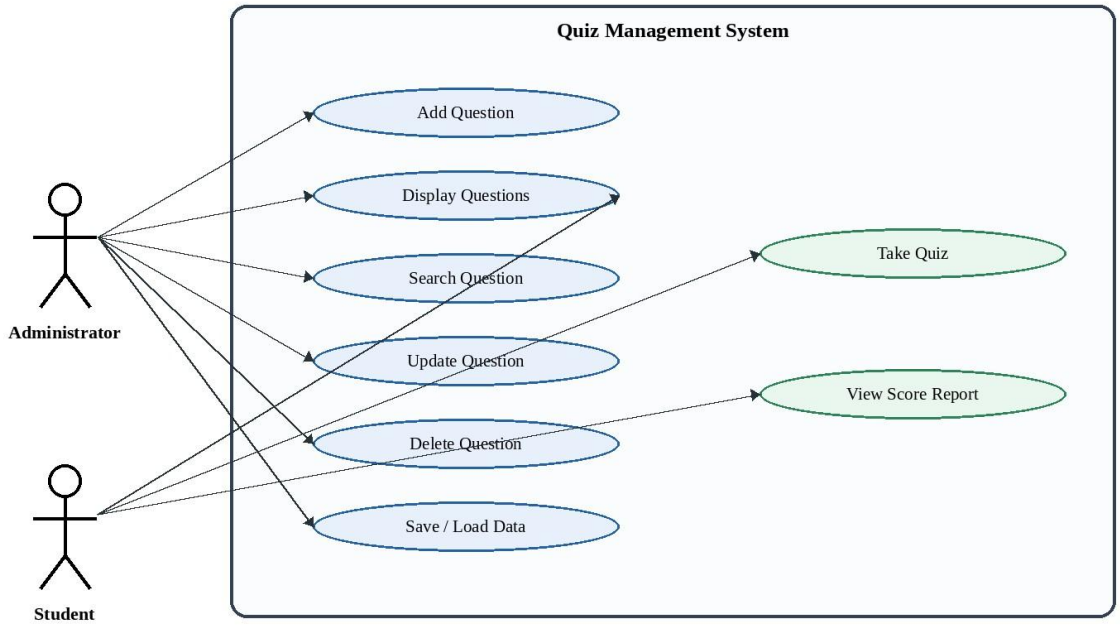
Add a question, select option 8 or exit using option 9, close the program, then restart it from the project root and display the question. To test attempt persistence, take a quiz and confirm that the attempt count and score are shown after restart. The final submission should include a screenshot from the student's own Windows run confirming this behavior.

9. UML Diagrams

The diagrams below represent the main actors, classes, and interactions. Sequence diagrams illustrate conceptual message flow for the two important operations.

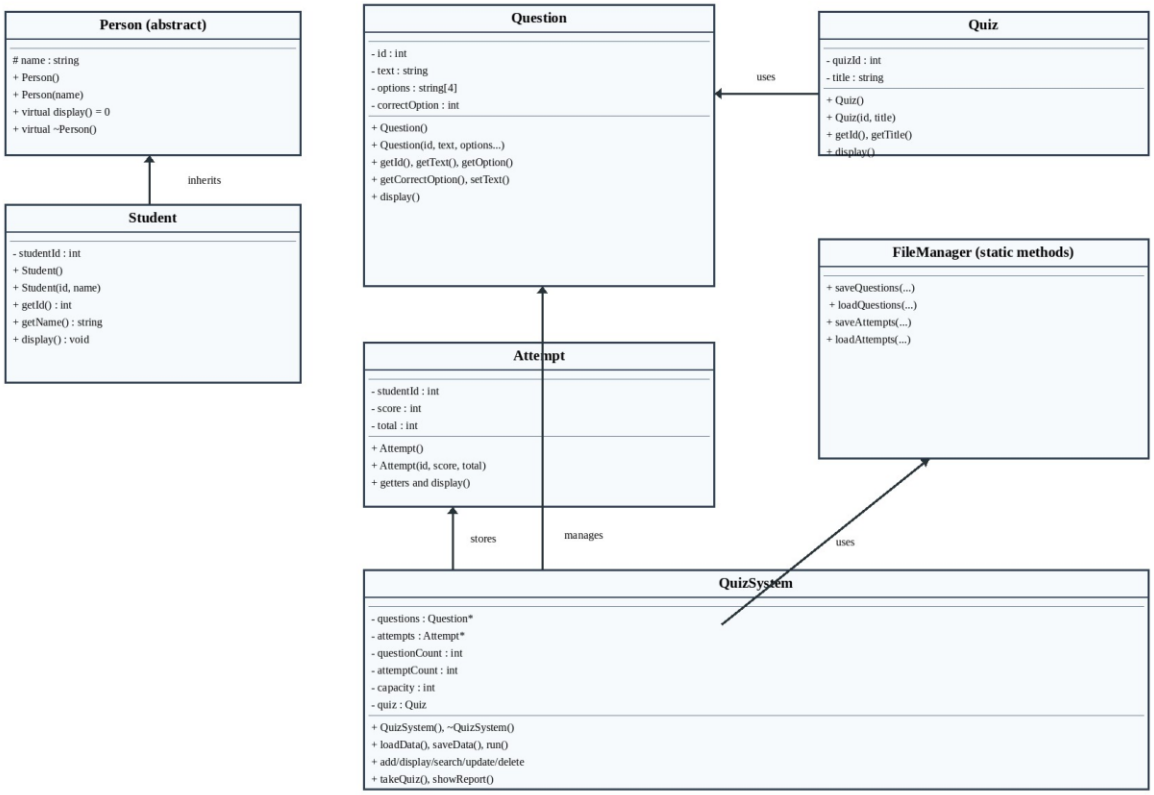
9.1 Use Case Diagram

USE CASE DIAGRAM — QUIZ MANAGEMENT SYSTEM

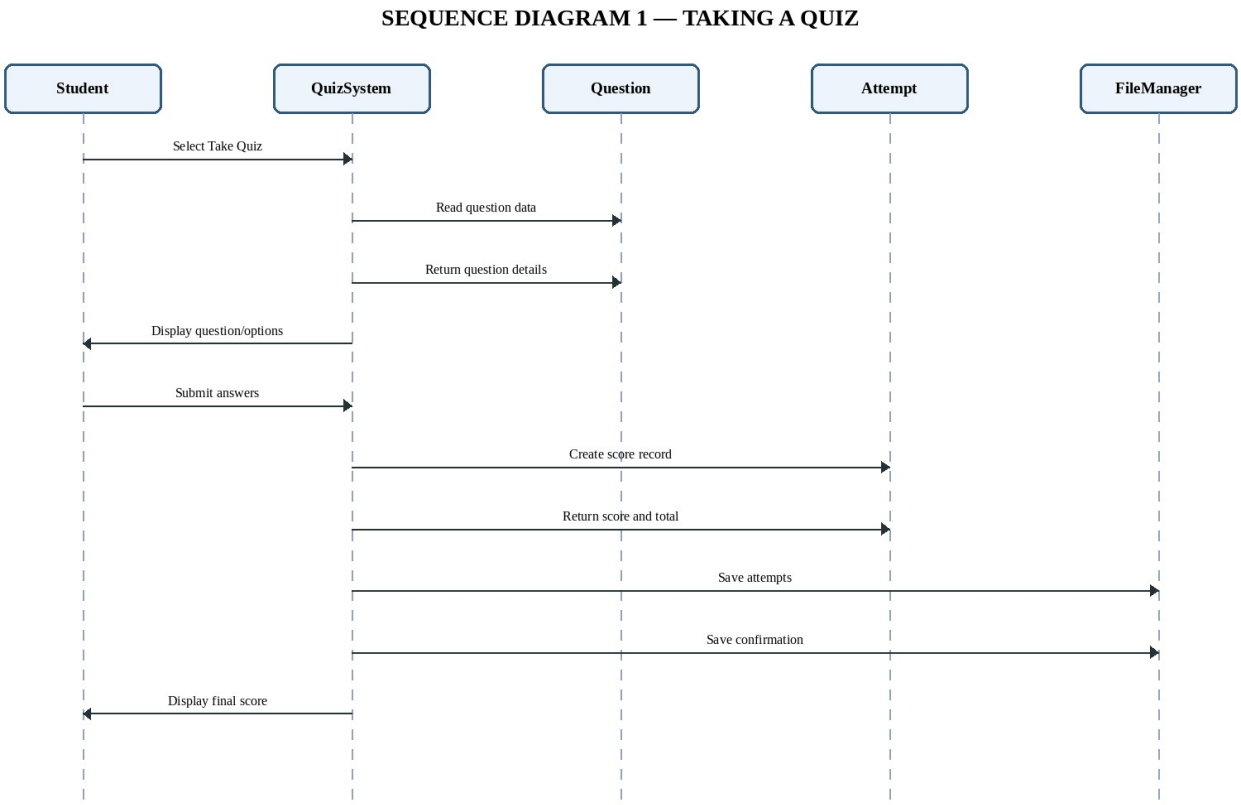


9.2 Class Diagram

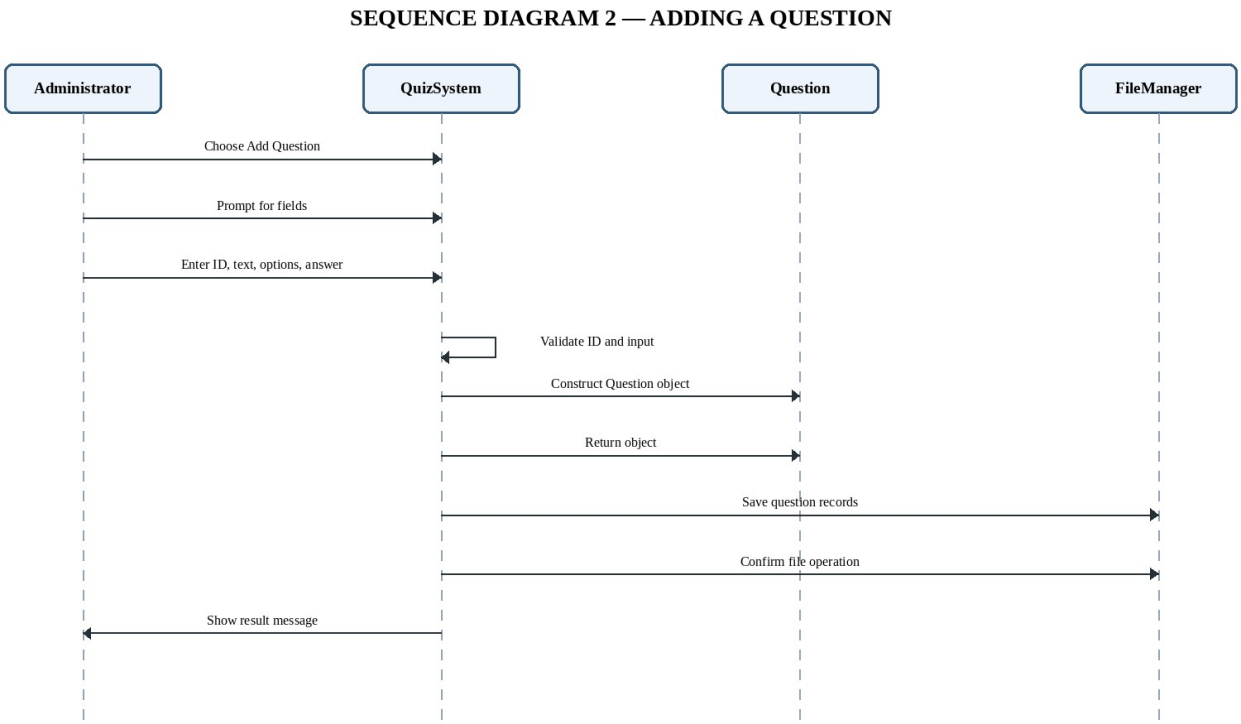
CLASS DIAGRAM — QUIZ MANAGEMENT SYSTEM



9.3 Sequence Diagram 1 — Taking a Quiz



9.4 Sequence Diagram 2 — Adding a Question



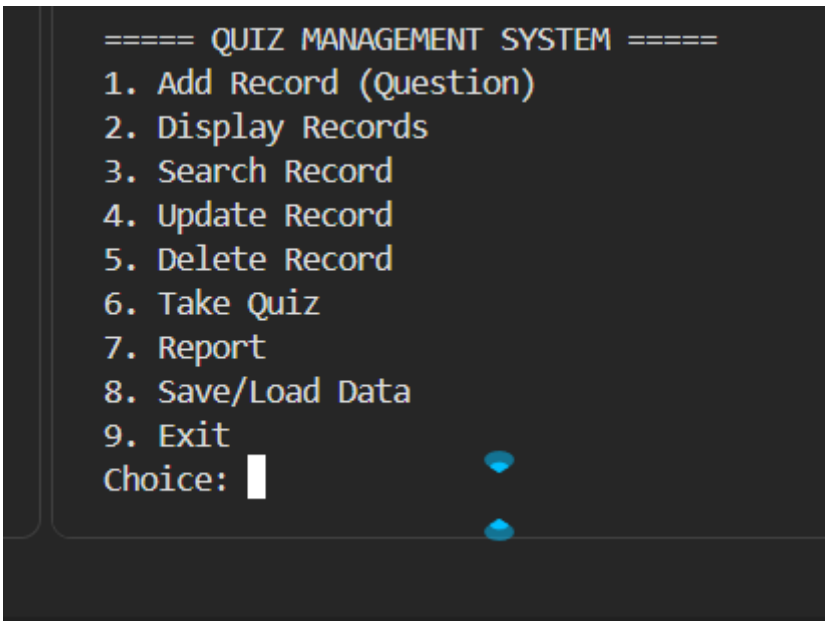
10. Program Workflow

- main() constructs a QuizSystem object.
- QuizSystem allocates storage for up to 100 questions and 100 attempts.
- loadData() loads saved records from the Data directory.
- The menu repeats until the user selects option 9.
- The selected operation manages questions, conducts a quiz, displays a report, or saves and reloads data.
- During a quiz, the program collects student ID and name, displays each question, accepts a numbered answer, and increments the score when the answer matches the correct option.
- An Attempt record stores student ID, score, and total questions if attempt storage is not full.
- On exit, the system saves the records; the destructor releases the dynamically allocated arrays.

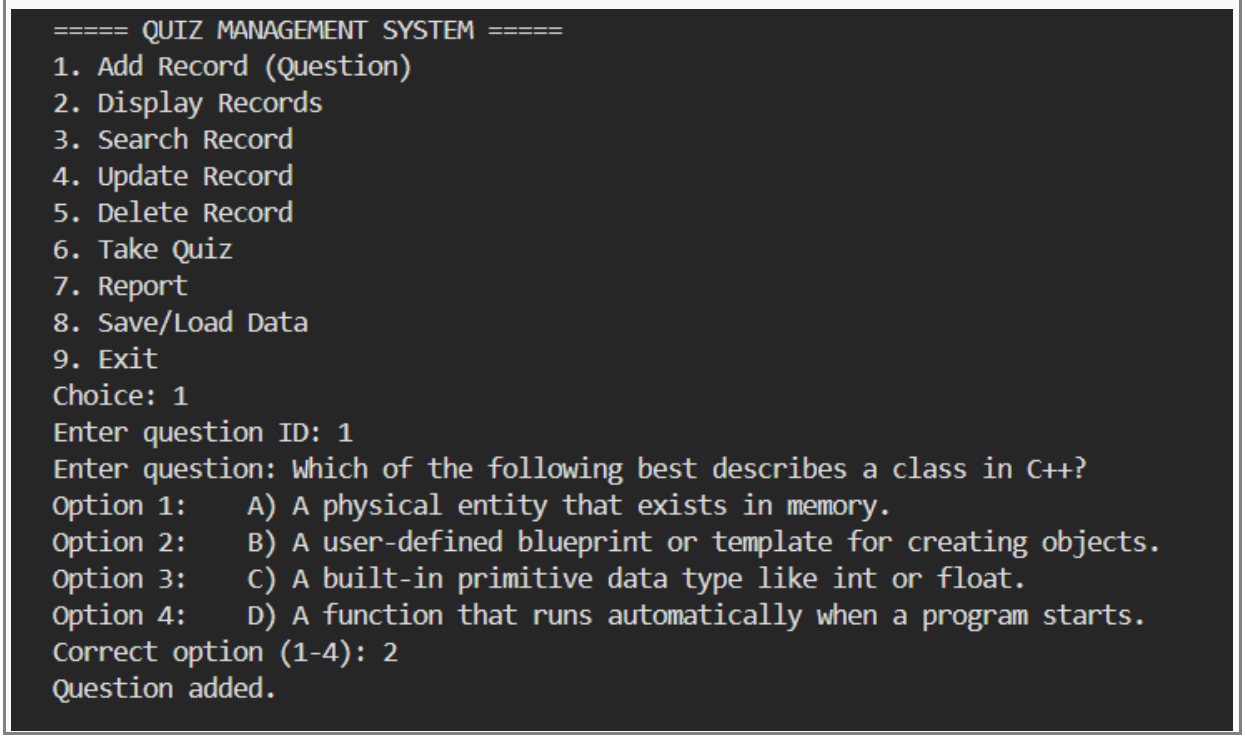
11 . Screenshots

Screenshots from the actual target environment were not included in the available project files. To avoid presenting invented screenshots as genuine execution evidence, the following spaces are placeholders. Replace them with screenshots captured after compiling and running the program on the student's Windows computer.

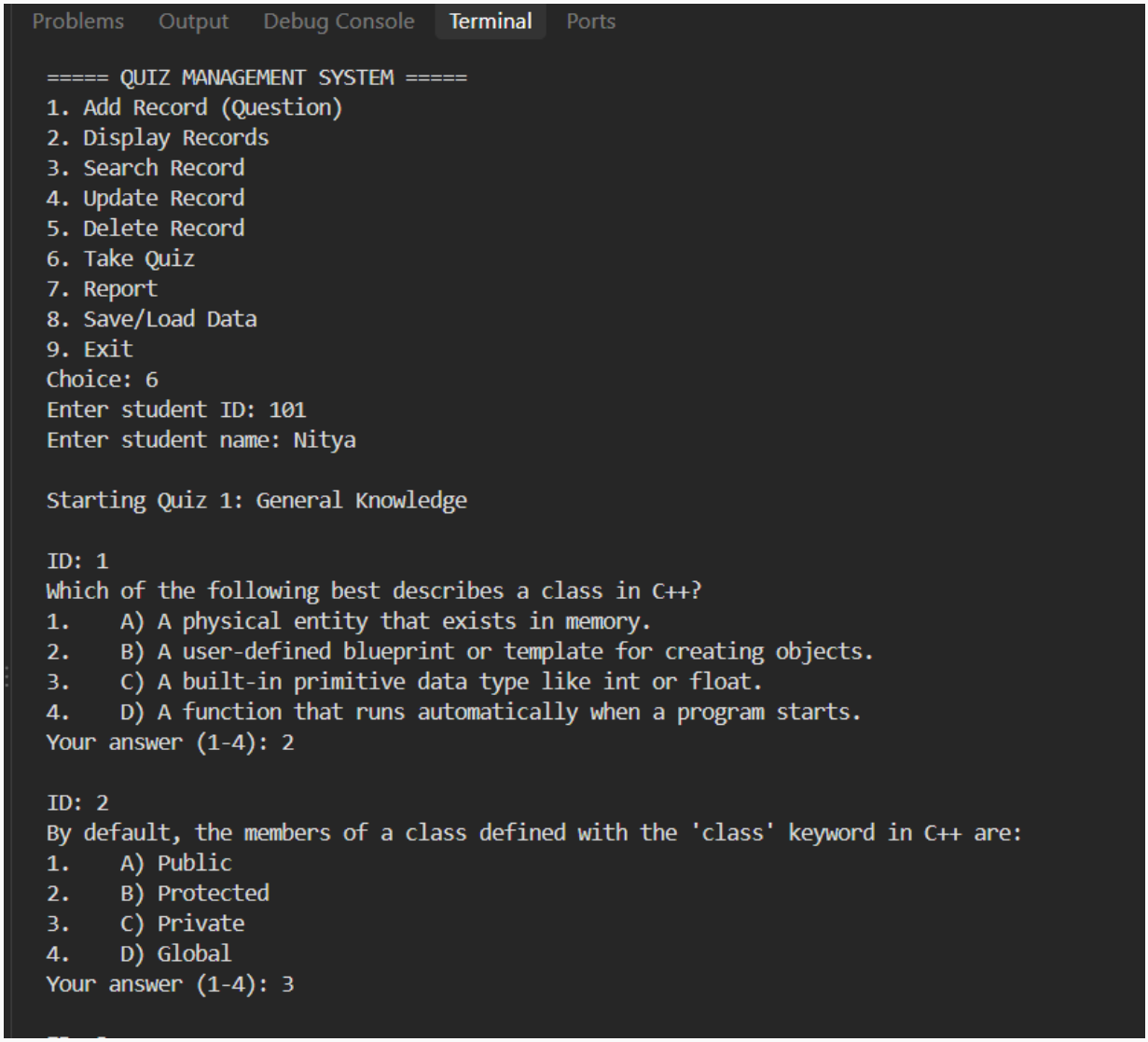
Screenshot 1 — Main Menu



Screenshot 2 — Question Management



Screenshot 3 — Quiz and Result



```
Go  Run  Terminal  Help        QuizManagementSystem

Problems  Output  Debug Console  Terminal  Ports

 Your answer (1-4): 2

 ID: 10
  Which standard stream object is typically used for standard console output in C++?
1.      A) cin
2.      B) clog
3.      C) cout
4.      D) cerr
 Your answer (1-4): 3
 Quiz completed. Score: 7/10
```

Screenshot 4 — Persistence / Report

```
===== QUIZ MANAGEMENT SYSTEM =====
1. Add Record (Question)
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Take Quiz
7. Report
8. Save/Load Data
9. Exit
Choice: 7

--- Quiz Report ---
Total questions: 10
Total attempts: 4
Student ID: 102 | Score: 1/1
Student ID: 101 | Score: 10/10
Student ID: 0 | Score: 2/10
Student ID: 102 | Score: 7/10
```

12. Testing

The following test cases are based on the implemented menu. Expected results describe the behavior intended by the source code. The actual result column is deliberately marked for verification because a Windows execution log was not supplied.

Test	Input / action	Expected output	Actual result
TC-01	Start with empty data files.	Menu appears; counts are zero.	Verify on Windows.
TC-02	Add a valid question with a unique ID.	“Question added.”	Verify on Windows.
TC-03	Add a duplicate question ID.	“ID already exists.”	Verify on Windows.
TC-04	Use empty question text or answer outside 1 – 4.	Validation message; not record added.	Verify on Windows.
TC-05	Search for an existing ID.	Question and options displayed.	Verify on Windows.
TC-06	Search for a missing ID.	“Question not found.”	Verify on Windows.
TC-07	Update existing question.	Question text changes; options/answer remain unchanged.	Verify on Windows.
TC-08	Delete existing question.	“Question deleted.”	Verify on Windows.
TC-09	Take a quiz and answer questions.	Score displayed as correct / total; attempt recorded if capacity remains.	Verify on Windows.
TC-10	Save, exit, and restart.	Saved records load from Data files.	Verify on Windows.
TC-11	Enter a non-numeric menu choice.	Program requests a number from 1 to 9.	Verify on Windows.

13. Challenges and Solutions

Challenge	Solution / current approach
Keeping code maintainable	Split the program into classes and separate .h/.cpp files.
Avoiding duplicate IDs	Search existing questions before adding a new one.
Retaining records after exit	Use text files; load at startup and save on request and exit.
Managing arrays	Allocate with new[] and release with delete[] in the destructor.
Handling invalid input and file errors	Use basic validation, input clearing, and try/catch.
Cross-platform execution	Compile the source with a Windows-compatible compiler; a Linux executable is not a native Windows .exe.

Known limitations: update changes only question text; attempts store student ID and score but not student name or date/time; capacity is limited to 100 records; pipe characters are not escaped in fields; and some numeric input paths have limited validation. These can be addressed in a future version.

14. Conclusion

The Quiz Management System demonstrates how a menu-driven C++ program can manage multiple-choice questions, conduct quizzes, calculate scores, generate a simple report, and retain data in text files. It applies classes and objects, encapsulation, constructors and destructors, inheritance, virtual functions, dynamic memory allocation, file handling, and basic exception handling. Separating responsibilities across classes makes the program easier to understand and maintain. Before final submission, compile and test the application on the target Windows system, replace screenshot placeholders with genuine execution screenshots, and verify that diagrams match the final source code.

Appendix: Build and Run Notes

Open the project folder in Visual Studio Code. Ensure a C++ compiler such as MinGW-w64 is installed and available in PATH. Run BUILD_WINDOWS.bat from the project root if the file is included in the project package. Keep the Data directory alongside the project and launch the application from the project root so relative file paths work. The Linux executable in the supplied package cannot run as a native Windows .exe.

Formatting note: body text uses 12-point Liberation Serif, a Times-compatible serif font, with justified alignment. Headings and table text use the same 12-point font family, with bold styling where appropriate.